

Simulación Termohidráulica de Tiempo Real en Rust

*Arquitectura de Componentes, Modelos Matemáticos y Código Fuente
para Sistemas 0D/1D Distribuidos*

Preparado para:
Biblioteca de Componentes Físicos
Orquestación por Diagrama de Bloques

Lenguaje de Implementación:
Rust (Edición 2021)
Rendimiento Monofásico Explícito

Índice

1. Componentes de Conexión y Distribución de Flujo (Resistencias)	4
1.1. Modelo Termohidráulico de Tubería 1D Discretizada	4
1.1.1. Topología del Modelo (Grilla Escalonada)	4
1.1.2. Estructura de Datos y Caché (SoA)	4
1.1.3. Estabilización Numérica y Límites	5
1.1.4. Implementación del Motor de Simulación (Método tick)	5
1.2. Bomba Centrífuga Dinámica	7
1.2.1. El Modelo Matemático de la Bomba Dinámica	7
1.2.2. Implementación de la Lookup Table 1D Lineal en Rust	7
1.2.3. Estructura e Implementación del Bloque Bomba	8
1.2.4. Acoplamiento de Energía (Transporte Térmico)	9
2. Componentes Volumétricos de Acumulación (Capacitancias)	10
2.1. Bloque Header (Volumen de Control Capacitivo)	10
2.1.1. Rol Arquitectónico y Topología	10
2.1.2. Modelado Matemático y Conservación de Masa	10
2.1.3. Estructura de Datos en Rust	11
2.1.4. Implementación del Bloque (Método tick)	11
2.1.5. Acoplamiento de Frontera y Flujo Bidireccional	12
2.2. Open Tank (Tanque de Presión Hidrostática)	13
2.2.1. El Modelado Matemático	13
2.2.2. Estructura de Datos en Rust	13
2.2.3. Implementación del Bloque (tick)	14
2.2.4. Particularidades en la Red	15
2.3. Tank Closed (Tanque Presurizado de Nivel Variable)	16
2.3.1. El Modelado Matemático (0D de Dos Zonas)	16
2.3.2. Implementación del Bloque (tick)	17
2.4. Tanque Térmico Estratificado	18
2.4.1. Topología de N Capas ($N = 2, 3, 4, \dots$)	18
2.4.2. Modelado de los Fenómenos de Mezclado	18
2.4.3. Implementación en Rust (Modelo de 3 Capas Conservativo)	19
3. API de Propiedades Termodinámicas	21
3.1. Diseño de la API Termodinámica (thermo-api)	21
3.1.1. Estructura de Estado Termodinámico	21
3.1.2. La Trait ThermoLibrary	21
3.1.3. Implementaciones Previstas	22
3.1.4. Uso en los Bloques de Simulación	22
4. Ejemplos de Sistemas Complejos	23
4.1. Ejemplo: Circuito de Control Térmico Industrial	23
4.1.1. Esquema del Sistema	23
4.1.2. Componentes y Parámetros	23
4.1.3. Lazo de Control	24
4.1.4. Estabilidad Numérica	24
5. Arquitectura de Integración con el Motor de Bloques	25
5.1. Integración con el Motor de Diagramas de Bloques	25
5.1.1. El Paradigma de Acoplamiento (Esfuerzo-Flujo)	25

5.1.2. Mapeo de Puertos en el Sistema bloques	25
5.1.3. Orden de Ejecución y Estabilidad Numérica	26
5.1.4. Manejo de la Red de Presión (Algebraic Loops)	26
5.1.5. Sincronización Termodinámica Global	26

1. Componentes de Conexión y Distribución de Flujo (Resistencias)

1.1. Modelo Termohidráulico de Tubería 1D Discretizada

Este documento detalla la arquitectura, el modelado matemático y la implementación en Rust de un componente de tubería unidimensional (Pipe1D) basado en volúmenes finitos. El modelo está diseñado para simulaciones de tiempo discreto de fluidos monofásicos (como sistemas de refrigeración de reactores o circuitos de agua pesada), priorizando la estabilidad numérica, la velocidad de ejecución por ciclo y el manejo natural de flujos bidireccionales y de bajo caudal.

1.1.1. Topología del Modelo (Grilla Escalonada)

Para evitar la inestabilidad espacial que sufren los esquemas centrados, el Pipe1D utiliza una grilla escalonada (staggered grid). La tubería se divide en N volúmenes de control, lo que genera dos dominios de cálculo paralelos:

Celdas (N elementos): Representan los volúmenes finitos. Aquí residen las variables escalares: presión (p), entalpía (h) y densidad (ρ).

Caras o Uniones ($N + 1$ elementos): Representan las fronteras entre celdas (y los extremos conectados a los headers externos). Aquí reside la variable vectorial: el caudal másico (W).

Esta topología permite evaluar los gradientes de presión directamente sobre las caras, y los balances de masa directamente sobre las celdas.

1.1.2. Estructura de Datos y Caché (SoA)

```
pub struct Pipe1D {
    pub n_cells: usize,

    // Geometría (Parámetros fijos)
    vol_cell: f64,
    geom_inertia: f64, // A / dz
    fric_factor: f64, // (f * dz) / (2 * D * A^2)
    elevation_drop: f64, // z_in - z_out [m]

    // ...
}
```

1.1.3. Estabilización Numérica y Límites

⚠ Límite de Estabilidad CFL

Para que la integración de energía (advección) sea estable, el paso de tiempo debe ser menor al tiempo de tránsito de una celda:

$$\Delta t \leq \frac{\rho \cdot V_{\text{cell}}}{|W|}$$

Si el flujo es muy rápido ($|W|$ grande) o las celdas muy pequeñas (V_{cell} pequeño), el sistema oscilará. Se recomienda reducir el Δt global o aumentar el volumen de las celdas.

1.1.3.1. Momento: Fricción Semi-Implicita y Gravedad

Para habilitar la **convección natural**, la ecuación de momento se extiende con el término de cabezal hidrostático:

$$W^{n+1} = \frac{W^n + \frac{\Delta t \cdot A}{\Delta z} (\Delta P + \rho \cdot g \cdot \Delta z_{\text{cell}})}{1 + \frac{\Delta t \cdot A}{\Delta z} K_{\text{total}} \cdot \max(|W^n|, \varepsilon_w)}$$

Donde $\Delta z_{\text{cell}} = \frac{\text{elevation_drop}}{N}$. Si una rama del circuito está más caliente, su densidad ρ será menor, generando una diferencia de presión neta que impulsa el fluido sin necesidad de bombas.

1.1.4. Implementación del Motor de Simulación (Método tick)

```
impl PipeID {
    pub fn tick(&mut self, p_in: f64, h_in: f64, p_out: f64, h_out: f64, dt: f64) -
    > (f64, f64) {

        // 1. ECUACIÓN DE MOMENTO (N+1 Caras)
        let inertia_dt = self.geom_inertia * dt;

        for i in 0..self.n_cells {
            let p_up = if i == 0 { p_in } else { self.p[i-1] };
            let p_down = if i == self.n_cells { p_out } else { self.p[i] };
            let rho_f = if i == 0 { self.rho[0] }
                else if i == self.n_cells { self.rho[self.n_cells-1] }
                else { (self.rho[i-1] + self.rho[i]) * 0.5 };

            self.w[i] = self.calc_flow_semi_implicit(self.w_last[i], p_up, p_down,
            rho_f, inertia_dt);
        }

        // 2. BALANCES CONSERVATIVOS (N Celdas)
        for i in 0..self.n_cells {
            let phi_in = self.w[i] * self.calc_h_face_smooth(self.w[i],
                if i == 0 { h_in } else { self.h[i-1] }, self.h[i]);
            let phi_out = self.w[i+1] * self.calc_h_face_smooth(self.w[i+1],
                self.h[i], if i == self.n_cells - 1 { h_out } else
            { self.h[i+1] });
```

```

        self.m[i] += (self.w[i] - self.w[i+1]) * dt;
        self.u[i] += (phi_in - phi_out) * dt;

        let rho_real = self.m[i] / self.vol_cell;
        let u_spec = self.u[i] / self.m[i];

        let (p_new, h_new, _t_new) = thermo::get_state_from_rho_u(rho_real,
u_spec);

        self.p[i] = p_new;
        self.h[i] = h_new;
        self.rho[i] = rho_real;
    }

    self.w_last.copy_from_slice(&self.w);
    (self.w[0], self.w[self.n_cells])
}

#[inline(always)]
fn calc_flow_semi_implicit(&self, w_prev: f64, p_up: f64, p_down: f64, rho:
f64, inertia_dt: f64) -> f64 {
    let delta_p = p_up - p_down;
    let numerator = w_prev + inertia_dt * delta_p;
    let w_mod = f64::max(w_prev.abs(), 1e-4);
    let denominator = 1.0 + inertia_dt * (self.fric_factor / rho) * w_mod;
    numerator / denominator
}

#[inline(always)]
fn calc_h_face_smooth(&self, w: f64, h_up: f64, h_down: f64) -> f64 {
    let epsilon_w = 1e-4;
    if w > epsilon_w {
        h_up
    } else if w < -epsilon_w {
        h_down
    } else {
        let factor = (w + epsilon_w) / (2.0 * epsilon_w);
        factor * h_up + (1.0 - factor) * h_down
    }
}
}

```

1.2. Bomba Centrífuga Dinámica

Modelar una bomba centrífuga en tu esquema de tiempo discreto es sumamente elegante porque, desde el punto de vista topológico, la bomba se comporta exactamente igual que una cara (o junction) del caño: es una resistencia activa al flujo.

En lugar de que el motor del caudal sea únicamente la diferencia de presiones de los headers ($\Delta P = P_{\text{in}} - P_{\text{out}}$), la bomba añade un término de altura manométrica o salto de presión activo (ΔP_{bomba}) que depende del caudal actual y de la velocidad de giro de la bomba (N), determinado por tu lookup table.

Aquí tienes el diseño físico y la implementación en Rust para acoplarla de forma estable y bidireccional.

1.2.1. El Modelo Matemático de la Bomba Dinámica

La ecuación de momento discreta para el bloque bomba se deriva modificando la ecuación del caño. El salto de presión total que ve el fluido es el gradiente de los headers más la presión que inyecta el rodete, menos las pérdidas internas de la bomba:

$$W^{n+1} = W^n + \frac{\Delta t \cdot A}{L} (P_{\text{in}} - P_{\text{out}} + \Delta P_{\text{bomba}}(W^n, N) - \text{Fricción}(W^n))$$

1.2.1.1. Las Leyes de Afinidad para la Lookup Table

Por lo general, el fabricante entrega la curva de la bomba como una tabla de datos de Altura (H en metros) versus Caudal Volumétrico (Q en $\frac{m^3}{h}$) a una velocidad de giro nominal (N_{nom}).

Si la simulación permite que la bomba varíe su velocidad (por ejemplo, controlada por un variador de frecuencia o por la inercia del motor en un transitorio de parada), usamos las Leyes de Afinidad de las bombas centrífugas para escalar la curva a cualquier velocidad N actual sin necesitar una tabla bidimensional gigante:

$$Q = Q_{\text{tab}} \cdot \left(\frac{N}{N_{\text{nom}}} \right) \Rightarrow W = W_{\text{tab}} \cdot \left(\frac{N}{N_{\text{nom}}} \right) \cdot \frac{\rho}{\rho_{\text{nom}}}$$

$$\Delta P_{\text{bomba}} = \Delta P_{\text{tab}} \cdot \left(\frac{N}{N_{\text{nom}}} \right)^2 \cdot \frac{\rho}{\rho_{\text{nom}}}$$

Esto significa que es posible usar una única lookup table estática 1D ($W_{\text{tab}} \Rightarrow \Delta P_{\text{tab}}$). Para cualquier caudal real W , se «normaliza» a la velocidad nominal, se busca en la tabla la presión nominal, y luego se escala el resultado.

1.2.2. Implementación de la Lookup Table 1D Lineal en Rust

Para mantener las miles de consultas por segundo, la lookup table debe hacer una búsqueda binaria rápida e interpolación lineal, evitando cualquier asignación de memoria dinámica (alloc) en el loop.

```
pub struct LookupTable1D {
    x: Vec<f64>, // Caudales máxicos de prueba (deben estar ordenados)
    y: Vec<f64>, // Saltos de presión correspondientes (Pa)
}

impl LookupTable1D {
```

```

pub fn new(x: Vec<f64>, y: Vec<f64>) -> Self {
    assert_eq!(x.len(), y.len());
    Self { x, y }
}

/// Interpolación lineal con extrapolación segura en los extremos
pub fn interpolate(&self, target_x: f64) -> f64 {
    let n = self.x.len();
    if target_x <= self.x[0] { return self.y[0]; }
    if target_x >= self.x[n - 1] { return self.y[n - 1]; }

    // Búsqueda binaria optimizada (O(log N))
    let idx = match self.x.binary_search_by(|val|
val.partial_cmp(&target_x).unwrap()) {
        Ok(exact_idx) => exact_idx,
        Err(insert_idx) => insert_idx - 1,
    };

    // Interpolación lineal estándar
    let x0 = self.x[idx];
    let x1 = self.x[idx + 1];
    let y0 = self.y[idx];
    let y1 = self.y[idx + 1];

    y0 + (target_x - x0) * (y1 - y0) / (x1 - x0)
}
}

```

1.2.3. Estructura e Implementación del Bloque Bomba

```

pub struct CentrifugalPumpBlock {
    // Geometría interna (Independiente del dt)
    geom_inertia: f64, // A / L

    // Curva característica a velocidad nominal
    curve_table: LookupTable1D,
    n_nominal: f64,
    rho_nominal: f64,

    fric_factor_pasivo: f64,
    w_last: f64,
}

impl CentrifugalPumpBlock {
    pub fn tick(&mut self, p_in: f64, p_out: f64, rho: f64, speed: f64, dt: f64) -> f64 {
        let delta_p_headers = p_in - p_out;
        let inertia_dt = self.geom_inertia * dt;

        let speed_ratio = speed / self.n_nominal;
        let rho_ratio = rho / self.rho_nominal;
    }
}

```



```

let dp_bomba = if speed_ratio.abs() > 1e-3 && self.w_last >= 0.0 {
    let w_nominal_look = self.w_last / (speed_ratio * rho_ratio);
    let dp_nominal_look = self.curve_table.interpolate(w_nominal_look);
    dp_nominal_look * speed_ratio.powi(2) * rho_ratio
} else {
    0.0
};

// --- ECUACIÓN DE MOMENTO SEMI-IMPLÍCITA ---
let numerator = self.w_last + inertia_dt * (delta_p_headers + dp_bomba);

let w_mod = f64::max(self.w_last.abs(), 1e-4);
let k_fric = if self.w_last >= 0.0 { self.fric_factor_pasivo } else
{ self.fric_factor_pasivo * 5.0 };
let denominator = 1.0 + inertia_dt * (k_fric / rho) * w_mod;

let w_current = numerator / denominator;
self.w_last = w_current;
w_current
}
}

```

1.2.4. Acoplamiento de Energía (Transporte Térmico)

La bomba genera un trabajo sobre el fluido que incrementa su energía. Al igual que con el caño, el transporte de entalpía hacia los headers se calcula afuera de la ecuación de momento usando el Upwind Suavizado:

- **Flujo Directo ($W > 0$):** La bomba succiona entalpía del Header_In y se la inyecta al Header_Out. Físicamente, parte de la potencia consumida por la bomba que no se convierte en presión (ineficiencia térmica o fricción interna) se disipa directamente como calor en el fluido. Se puede sumar esa ineficiencia como un término $Q_{\text{disipado}} = \text{Potencia} \cdot (1 - \eta)$ directo al flujo de energía que recibe el Header_Out.
- **Flujo Inverso ($W < 0$):** Si las presiones externas vencen a la bomba y el fluido retrocede, la entalpía viaja desde el Header_Out hacia el Header_In.

2. Componentes Volumétricos de Acumulación (Capacitancias)

2.1. Bloque Header (Volumen de Control Capacitivo)

Este documento detalla la arquitectura, el modelado matemático y la implementación en Rust del componente Header (o nodo volumétrico de acumulación). En una simulación termohidráulica por diagramas de bloques distribuidos, el Header actúa como una capacitancia acoplada. Es el encargado de absorber los cambios bruscos de caudal impuestos por las resistencias (Pipe1D, válvulas, bombas), actuando como un amortiguador numérico que «ablanda» las ecuaciones de presión y energía del sistema discreto.

2.1.1. Rol Arquitectónico y Topología

A diferencia del caño discretizado espacialmente, el Header es un modelo de parámetros concentrados (0D).

Variables de Estado: Es el dueño de la presión (p), la entalpía (h) y la densidad (ρ) en un punto de conexión común.

Conectividad: Funciona como un nodo central («hub») al que se pueden conectar M componentes de flujo. No presupone una dirección única; recolecta caudales de masa (W) y flujos de energía (Φ) netos provenientes de todas sus fronteras conectadas, aplicando una convención de signos estricta: flujo entrante es positivo (+), flujo saliente es negativo (-).

2.1.2. Modelado Matemático y Conservación de Masa

Para garantizar la estabilidad y la precisión física, el Header utiliza un enfoque de **variables conservativas**. Las variables de estado primarias que se integran en el tiempo son la masa total (M) y la energía interna total (U).

2.1.2.1. Ecuaciones de Conservación

Las ecuaciones continuas que gobiernan el volumen fijo V del Header son:

$$\frac{dM}{dt} = \sum W_{\text{in}} - \sum W_{\text{out}} = W_{\text{net}}$$

$$\frac{dU}{dt} = \sum (W \cdot h)_{\text{in}} - \sum (W \cdot h)_{\text{out}} + Q = \Phi_{\text{net}} + Q$$

A partir de estas, calculamos las propiedades intensivas en cada paso:

1. **Densidad:** $\rho = \frac{M}{V}$
2. **Energía específica:** $u = \frac{U}{M}$
3. **Presión y Temperatura:** $p, T = f(\rho, u)$ (usando la ecuación de estado).

2.1.2.2. Estrategia de Estabilización Numérica (Ablandamiento de Presión)

En simulaciones de tiempo real con líquidos casi incompresibles, un pequeño error en el balance de masa genera oscilaciones masivas de presión. Para «ablandar» este acoplamiento sin violar la conservación de masa, se utiliza un método de **compresibilidad virtual** en la ecuación de estado:

$$dp = \frac{1}{V \cdot \left(\frac{\partial \rho}{\partial p}\right)_{\text{virtual}}} \left[\Delta M - V \cdot \left(\frac{\partial \rho}{\partial h}\right)_p \Delta h \right]$$

Donde $\left(\frac{\partial \rho}{\partial p}\right)_{\text{virtual}} = \beta \cdot \left(\frac{\partial \rho}{\partial p}\right)_{\text{real}}$ con $\beta \approx 10..100$. Es vital que este ablandamiento sea consistente: la densidad utilizada en el siguiente paso de transporte debe ser la derivada de la masa real, no de la presión ablandada, para evitar derivas de masa.

2.1.3. Estructura de Datos en Rust

```
pub struct HeaderBlock {
    volume: f64,

    // VARIABLES DE ESTADO CONSERVATIVAS
    pub mass: f64,
    pub internal_energy: f64,

    // SALIDAS CALCULADAS
    pub p: f64,
    pub h: f64,
    pub rho: f64,

    // PARÁMETROS DE ESTABILIDAD
    beta_softening: f64,
}
```

2.1.4. Implementación del Bloque (Método tick)

```
impl HeaderBlock {
    pub fn tick(&mut self, w_net: f64, wh_net: f64, q_externo: f64, dt: f64) {
        // 1. INTEGRACIÓN DE VARIABLES CONSERVATIVAS (Masa y Energía)
        self.mass += w_net * dt;
        self.internal_energy += (wh_net + q_externo) * dt;

        // Evitar estados no físicos por errores numéricos
        if self.mass < 1e-6 { self.mass = 1e-6; }

        // 2. ACTUALIZACIÓN TERMODINÁMICA
        self.refresh_thermo_properties();
    }

    fn refresh_thermo_properties(&mut self) {
        // Propiedades intensivas reales
        self.rho = self.mass / self.volume;
        let u = self.internal_energy / self.mass;

        // Enlace a librería compartida (IAPWS-IF97)
        // Nota: p y h deben ser calculadas de forma consistente con rho y u
        let (p_real, h_real, t_real) = thermo::get_state_from_rho_u(self.rho, u);

        // Aplicación del ablandamiento para el siguiente paso de presión
        // que verán los caños (esto reduce el golpe de ariete numérico)
    }
}
```

```
        self.p = p_real; // Opcional: aplicar filtro paso bajo si beta > 1
        self.h = h_real;
    }
}
```

2.1.5. Acoplamiento de Frontera y Flujo Bidireccional

El Header no necesita conocer qué componentes están conectados a él. Su interfaz expone de forma pública sus tres salidas actualizadas: p , h y ρ . La bidireccionalidad se resuelve en las conexiones gracias al esquema de transporte Upwind implementado en los caños:

Si un caño genera un caudal hacia afuera del Header, el caño lee la entalpía h del Header para calcular la energía que se está llevando.

Si un caño empuja fluido hacia adentro del Header, el caño le inyectará su propia entalpía de salida. El Header recibirá un w_{net} positivo y un wh_{net} positivo, absorbiendo la masa y mezclando la energía de forma automática.

2.2. Open Tank (Tanque de Presión Hidrostática)

Un tanque abierto (o pileta de nivel variable) es la simplificación más noble del bloque volumétrico. Al estar en contacto directo con la atmósfera, la cámara de gas desaparece de las ecuaciones porque su presión ya no depende de la compresión del volumen: la presión en la superficie es una constante fija (P_{atm}).

Esto significa que el colchón de aire ya no actúa como un resorte neumático. Toda la variación de presión en el fondo del tanque se debe única y exclusivamente al peso de la columna de líquido (la altura del nivel).

2.2.1. El Modelado Matemático

El modelo se reduce a la conservación de masa y al cálculo de la presión hidrostática pura.

2.2.1.1. Balance de Masa y Nivel

La masa acumulada (M_L) sigue gobernada por la suma algebraica de los caudales de los caños conectados (W_{net}):

$$M_L^{n+1} = M_L^n + W_{\text{net}} \cdot \Delta t$$

A partir de la masa y la densidad actual del líquido (ρ_L), determinamos el volumen y la altura del nivel (z):

$$V_L = \frac{M_L}{\rho_L}$$

$$z = \frac{V_L}{A}$$

2.2.1.2. Ecuación de Presión en la Base

La presión absoluta en el fondo del tanque que «ven» los caños acoplados es simplemente:

$$P_{\text{base}} = P_{\text{atm}} + \rho_L \cdot g \cdot z$$

i Info

En simulaciones numéricas, un tanque abierto es el estabilizador definitivo. Al fijar $P_{\text{surface}} = P_{\text{atm}}$, el sistema tiene un nodo de presión de referencia constante. Esto evita que las presiones del sistema floten o diverjan sin control numérico.

2.2.2. Estructura de Datos en Rust

```
pub struct OpenTankBlock {
    // Geometría fija
    area: f64,    // Área transversal (m²)
    g: f64,       // Gravedad (m/s²)
    p_atm: f64,   // Presión atmosférica local (Pa)

    // VARIABLES DE ESTADO INTEGRADAS
    pub m_liq: f64, // Masa de líquido actual (kg)
    pub h_liq: f64, // Entalpía específica del líquido (J/kg)
```

```
// SALIDAS DEDUCIDAS
pub p_base: f64, // Presión en el fondo (Pa)
pub level: f64,  // Nivel de líquido (m)
pub rho_l: f64,  // Densidad del líquido (kg/m³)
}
```

2.2.3. Implementación del Bloque (tick)

```
impl OpenTankBlock {
    pub fn new(area: f64, initial_level: f64, p_atm: f64, initial_h: f64) -> Self {
        let rho_l = 1000.0;
        let v_liq = initial_level * area;
        let m_liq = v_liq * rho_l;

        let mut tank = Self {
            area,
            g: 9.80665,
            p_atm,
            m_liq,
            h_liq: initial_h,
            p_base: p_atm + rho_l * 9.80665 * initial_level,
            level: initial_level,
            rho_l,
        };
        tank.refresh_geometry();
        tank
    }

    pub fn tick(&mut self, w_net: f64, wh_net: f64, q_externo: f64, dt: f64) {
        // 1. INTEGRACIÓN DE MASA
        self.m_liq += w_net * dt;
        if self.m_liq < 0.0 { self.m_liq = 0.0; }

        // 2. INTEGRACIÓN DE ENERGÍA
        let dh_dt = (wh_net + q_externo - (self.h_liq * w_net)) / (self.m_liq +
1e-3);
        self.h_liq += dh_dt * dt;

        // 3. ACTUALIZACIÓN DE SALIDAS
        self.refresh_geometry();
    }

    fn refresh_geometry(&mut self) {
        self.rho_l = 1000.0; // Enlace simplificado
        let v_liq = self.m_liq / self.rho_l;
        self.level = v_liq / self.area;
        self.p_base = self.p_atm + self.rho_l * self.g * self.level;
    }
}
```

2.2.4. Particularidades en la Red

Desborde Teórico: Si la masa inyectada supera la capacidad física, el modelo aumentará el level indefinidamente. Si se desea modelar el derrame, se debe limitar el level a la altura constructiva máxima en `refresh_geometry()` y reportar la masa perdida fuera del balance.

Evaporación y Pérdidas Térmicas (Q_{externo}): Los tanques abiertos tienen gran superficie de intercambio. El término Q_{externo} permite simular la pérdida de calor por convección con el aire o radiación solar.

2.3. Tank Closed (Tanque Presurizado de Nivel Variable)

Un tanque de nivel variable cerrado se diferencia del Header estructuralmente en un aspecto crítico: es un sistema de volumen de fluido variable con interfaz líquido-gas.

Mientras que el Header es un nodo totalmente lleno (monofásico cerrado, donde la presión sube instantáneamente al inyectar masa), el tanque cerrado absorbe masa cambiando su nivel de líquido, comprimiendo o expandiendo el colchón de gas (aire o nitrógeno) atrapado en la parte superior. Por lo tanto, la presión se rige por la ley de los gases ideales y procesos politrópicos.

2.3.1. El Modelado Matemático (0D de Dos Zonas)

Dividimos el tanque de volumen total fijo (V_{tot}) en dos zonas de volumen variable:

Zona Líquida (V_L): Fluido incompresible con densidad ρ_L .

Zona de Gas (V_G): Gas ideal compresible con masa fija M_G .

2.3.1.1. Balance de Masa y Geometría

La masa de líquido varía según el caudal neto de los caños conectados (W_{net}):

$$\frac{dM_L}{dt} = W_{\text{net}}$$

Conocida la masa de líquido, calculamos los volúmenes y el nivel hidrostático (z) si el tanque tiene un área transversal constante (A):

$$V_L = \frac{M_L}{\rho_L}$$

$$z = \frac{V_L}{A}$$

$$V_G = V_{\text{tot}} - V_L$$

2.3.1.2. Dinámica de Presión (Compresión del Gas)

Asumiendo que el gas se comprime de forma adiabática (proceso rápido sin transferencia de calor, $\gamma = 1.4$ para el aire) o isotérmica ($\gamma = 1.0$), la presión del gas P_G evoluciona como:

$$P_G \cdot V_G^\gamma = C \Rightarrow P_G = P_{G,0} \cdot \left(\frac{V_{G,0}}{V_G} \right)^\gamma$$

2.3.1.3. Dinámica de Energía (Balance Térmico con Trabajo de Presión)

Para el volumen de líquido, la evolución de la entalpía debe considerar no solo los flujos de masa sino también el trabajo realizado por el cambio de presión (especialmente relevante en tanques presurizados):

$$M_L \frac{dh_L}{dt} = \Phi_{\text{net}} + Q + V_L \frac{dp}{dt}$$

Donde $V_L \frac{dp}{dt}$ es el término de trabajo de flujo/compresión. En líquidos, este término suele ser pequeño pero es necesario para la consistencia energética total del sistema.

2.3.2. Implementación del Bloque (tick)

```
impl TankBlock {
    pub fn tick(&mut self, w_net: f64, wh_net: f64, q_externo: f64, dt: f64) {
        // 1. INTEGRACIÓN DE MASA
        self.m_liq += w_net * dt;

        // Límites físicos
        self.m_liq = self.m_liq.clamp(0.0, self.v_total * self.rho_l * 0.99);

        // 2. CÁLCULO DE PRESIÓN (Necesario para el término de trabajo)
        let p_base_old = self.p_base;
        self.refresh_geometry(); // Actualiza p_base, level, rho_l
        let dp_dt = (self.p_base - p_base_old) / dt;

        // 3. INTEGRACIÓN DE ENERGÍA (Incluyendo trabajo de presión)
        // Usamos la masa promediada o actual para estabilidad
        let work_term = (self.m_liq / self.rho_l) * dp_dt;
        let dh_dt = (wh_net + q_externo - (self.h_liq * w_net) + work_term) /
        (self.m_liq + 1e-3);
        self.h_liq += dh_dt * dt;
    }

    fn refresh_geometry(&mut self) {
        // Enlace a librería de propiedades compartida
        self.rho_l = thermo::get_rho_from_p_h(self.p_base, self.h_liq);

        let v_liq = self.m_liq / self.rho_l;
        let v_gas = self.v_total - v_liq;
        self.level = v_liq / self.area;

        // Ley politrópica del colchón de gas
        let p_gas = self.p_gas_nominal * (self.v_gas_nominal /
        v_gas).powf(self.gamma);
        self.p_base = p_gas + self.rho_l * self.g * self.level;
    }
}
```

2.4. Tanque Térmico Estratificado

Un tanque estratificado (típico en sistemas termosolares, acumulación térmica o agua caliente sanitaria) se basa en el principio de que el agua caliente es menos densa que el agua fría. Esto genera capas térmicas estables que no se mezclan de forma natural debido a la flotabilidad.

En un esquema de diagramas de bloques distribuidos, se modela como una serie de sub-volúmenes (nodos) acoplados verticalmente. Cada capa actúa matemáticamente como un Header o un pequeño tanque interconectado, pero aplicando dos leyes físicas adicionales: convección forzada (el agua se desplaza entre capas al entrar/salir fluido) y mezclado/conducción (intercambio destructivo de la estratificación).

2.4.1. Topología de N Capas ($N = 2, 3, 4...$)

Dividir el tanque en 3 o 4 capas suele ser el punto óptimo para simulación en tiempo real.

2 capas (Termoclina simple): Divide el tanque en «Capa Superior Hot» y «Capa Inferior Cold». Es rápido pero no captura bien los frentes térmicos móviles.

3 a 4 capas: Permite modelar una zona de transición (termoclina) que sube y baja según la carga y descarga del tanque.

```

+-----+
|  Capa 1 (Top / Hot)  | <- Entrada/Salida Alta (Caldera/Consumo)
+-----+
|      Capa 2          | | ^
+-----+               | | Flujos de acoplamiento interno (W_ij)
|      Capa 3          | | | Conducción térmica (Q_cond)
+-----+               | | v Mezclado flotante (Q_turb)
|  Capa 4 (Bottom/Cold) | <- Entrada/Salida Baja (Retorno)
+-----+

```

Cada capa i tiene su propia masa M_i , entalpía h_i y temperatura T_i . El volumen geométrico de cada capa es fijo $\left(V_i = \frac{V_{\text{tot}}}{N}\right)$.

2.4.2. Modelado de los Fenómenos de Mezclado

Hay tres mecanismos que transfieren energía entre la capa i y la capa $i + 1$:

2.4.2.1. Convección Forzada (Desplazamiento Neto de Masa)

Si se inyecta agua en la Capa 4 (abajo) y se extrae agua de la Capa 1 (arriba), se genera un caudal neto vertical ascendente (W_{net}) que atraviesa todas las capas intermedias. Cada capa le transfiere masa a la siguiente usando el esquema Upwind: si el flujo sube, la capa i recibe la entalpía de la capa $i + 1$.

2.4.2.2. Conducción Térmica Pasiva

Existe un flujo conductivo directo entre capas adyacentes debido a la diferencia de temperatura, que tiende a homogeneizar el tanque lentamente:

$$Q_{\text{cond}, i \rightarrow i+1} = \frac{k \cdot A}{\Delta x} (T_i - T_{i+1})$$

Donde k es la conductividad térmica del agua, A es el área transversal del tanque y Δx es la distancia entre los centros de las capas.

2.4.2.3. Mezclado por Inversión de Densidad (Penacho Convectivo)

Si $\rho_i > \rho_{i-1}$ (la capa de arriba es más densa que la de abajo), se produce una inestabilidad de Rayleigh-Taylor. En lugar de una conducción simple, modelamos un **caudal de intercambio turbulento** (W_{turb}) que mezcla ambas capas rápidamente:

$$W_{\text{turb}} = C_{\text{mix}} \cdot A \cdot \sqrt{g \cdot L_{\text{char}} \cdot \left(\Delta \frac{\rho}{\rho} \right)}$$

Este caudal extrae masa y energía de ambas capas y las promedia, simulando la caída de penachos fríos.

2.4.3. Implementación en Rust (Modelo de 3 Capas Conservativo)

```
pub struct StratifiedTank {
    volume_layer: f64,
    area: f64,
    dx: f64,
    k_water: f64,
    mix_factor: f64,

    // Variables de estado [0: Top, 1: Mid, 2: Bottom]
    // Integración de masa y energía por capa
    pub m: [f64; 3],
    pub u: [f64; 3],

    // Salidas calculadas
    pub p: f64,
    pub h: [f64; 3],
    pub t: [f64; 3],
    pub rho: [f64; 3],
}

impl StratifiedTank {
    pub fn tick(&mut self, w_top: f64, wh_top: f64, w_bot: f64, wh_bot: f64, dt: f64) {
        // 1. BALANCES DE MASA Y ENERGÍA
        let w_net = w_top;

        for i in 0..3 {
            // Flujos convectivos internos (entre capas, Upwind)
            // (Simplificado para ilustración)
            let _wh_01 = if w_net > 0.0 { w_net * self.h[1] } else { w_net * self.h[0] };

            // Intercambio por conducción y mezcla turbulenta
            let _q_mix_01 = self.calc_layer_exchange(0, 1);

            // Integración de m[i] y u[i]...
        }
    }
}
```

```

        self.refresh_properties();
    }

    fn calc_layer_exchange(&self, i: usize, j: usize) -> f64 {
        let q_cond = (self.k_water * self.area / self.dx) * (self.t[i] -
self.t[j]);

        // Si la densidad de la capa superior (i) es mayor que la inferior (j)
        if self.rho[i] > self.rho[j] {
            let w_turb = self.mix_factor * self.area * (self.rho[i] -
self.rho[j]).sqrt();
            let q_turb = w_turb * (self.h[i] - self.h[j]);
            return q_cond + q_turb;
        }
        q_cond
    }

    fn refresh_properties(&mut self) {
        for i in 0..3 {
            self.rho[i] = self.m[i] / self.volume_layer;
            let u_spec = self.u[i] / self.m[i];

            let (p, h, t) = thermo::get_state_from_rho_u(self.rho[i], u_spec);
            self.h[i] = h;
            self.t[i] = t;
        }
        // p se actualiza según la red de presión
    }
}

```

3. API de Propiedades Termodinámicas

3.1. Diseño de la API Termodinámica (thermo-api)

Para garantizar que todos los bloques de la simulación utilicen propiedades físicas consistentes y permitir el intercambio de motores de cálculo (ej. IAPWS-IF97 real vs. Tablas de búsqueda optimizadas), se propone la siguiente arquitectura de software en Rust.

3.1.1. Estructura de Estado Termodinámico

En lugar de solicitar propiedades individuales (lo que obligaría a recalcular la región de estado repetidamente), la API devuelve un objeto `ThermoState` que contiene todas las propiedades relevantes para el punto solicitado.

```
pub struct ThermoState {
    pub p: f64,          // Presión [Pa]
    pub t: f64,          // Temperatura [K]
    pub rho: f64,        // Densidad [kg/m³]
    pub h: f64,          // Entalpía específica [J/kg]
    pub u: f64,          // Energía interna específica [J/kg]

    // Propiedades de transporte (opcionales)
    pub cp: f64,         // Calor específico a P constante [J/kg·K]
    pub cv: f64,         // Calor específico a V constante [J/kg·K]

    // Derivadas parciales para estabilidad numérica (Jacobianos)
    pub drho_dp_h: f64, // ( $\partial\rho/\partial p$ )h -> Compresibilidad a entalpía constante
    pub drho_dh_p: f64, // ( $\partial\rho/\partial h$ )p -> Expansión térmica a presión constante
}
```

3.1.2. La Trait ThermoLibrary

Esta interfaz define el contrato que debe cumplir cualquier motor termodinámico.

```
pub trait ThermoLibrary: Send + Sync {
    /// Identificador del motor (ej. "IAPWS-IF97", "WaterTable-100x100")
    fn name(&self) -> &str;

    /// Cálculo desde variables intensivas estándar (Presión, Entalpía)
    fn from_p_h(&self, p: f64, h: f64) -> ThermoState;

    /// Cálculo desde variables conservativas (Densidad, Energía Interna)
    /// Este es el método principal usado por los Headers y Celdas 1D.
    fn from_rho_u(&self, rho: f64, u: f64) -> ThermoState;

    /// Cálculo desde condiciones de borde (Presión, Temperatura)
    fn from_p_t(&self, p: f64, t: f64) -> ThermoState;
}
```

3.1.3. Implementaciones Previstas

1. **IF97Backend**: Utiliza la librería `seuif97` para cálculos exactos de agua/vapor según el estándar industrial. Ideal para validación y transitorios lentos donde la precisión es crítica.
2. **TableBackend**: Utiliza una grilla precalculada (Lookup Table) con interpolación bilineal o bicúbica. Es el motor recomendado para producción y tiempo real por su tiempo de ejecución constante ($O(1)$) y predecible.
3. **LinearBackend**: Modelo de fluido incompresible con propiedades constantes (ej. $\rho = 1000$, $C_p = 4184$). Útil para pruebas rápidas y depuración de la lógica de control.

3.1.4. Uso en los Bloques de Simulación

Los bloques no instancian su propia librería, sino que reciben una referencia a una `dyn ThermoLibrary` a través del contexto de simulación o como un recurso inyectado.

```
impl SimulationBlock for HeaderBlock {
    fn tick(&mut self, ..., ctx: &Context) {
        // ... integración de m y u ...
        let thermo = ctx.get_thermo();
        let state = thermo.from_rho_u(self.mass / self.volume, self.u_total /
self.mass);

        self.p = state.p;
        self.h = state.h;
        self.rho = state.rho;
    }
}
```

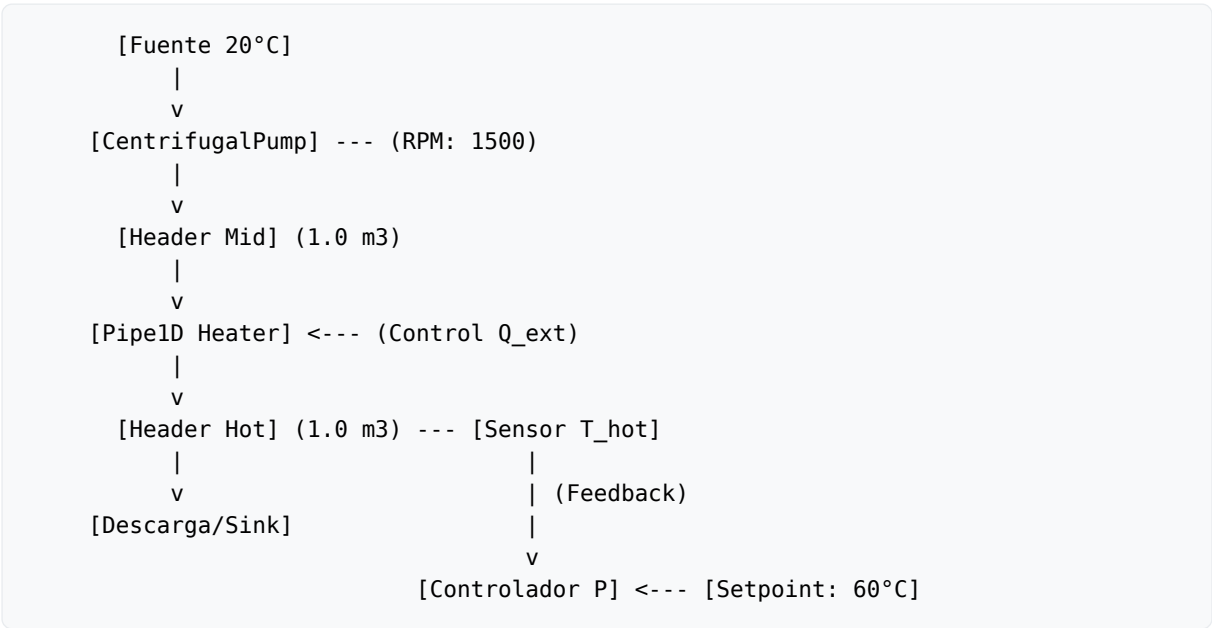
Optimización: El `TableBackend` es especialmente potente porque las derivadas parciales como `drho_dp_h` se pueden precalcular y almacenar directamente en la tabla, eliminando la necesidad de diferenciación numérica en tiempo de ejecución.

4. Ejemplos de Sistemas Complejos

4.1. Ejemplo: Circuito de Control Térmico Industrial

Este ejemplo demuestra la capacidad de la librería para modelar un sistema completo que integra hidráulica, termodinámica y control automático. Se simula un lazo de enfriamiento/calentamiento donde una bomba impulsa fluido a través de una tubería calefaccionada bajo un lazo de control proporcional.

4.1.1. Esquema del Sistema



4.1.2. Componentes y Parámetros

4.1.2.1. 1. Fuentes de Borde

- **Presión de Succión:** Constant 1.0 bar (10^5 Pa).
- **Temperatura de Entrada:** Constant 20°C.

4.1.2.2. 2. Bomba Centrífuga (CentrifugalPump)

- **Curva de Presión:** $dP \text{ [Pa]} = 0.2222 \cdot n^2 - 20 \cdot w^2$
- **Inercia Geométrica:** 0.01 m.
- **Fricción Pasiva:** 0.1.
- **Velocidad:** 1500 RPM.

4.1.2.3. 3. Tubería de Calentamiento (Pipe1D)

- **Discretización:** 5 celdas.
- **Geometría:** 5.0m de longitud, 0.1m de diámetro.
- **Calor Externo:** Recibe la señal del controlador (vatios).

4.1.2.4. 4. Nodos de Acumulación (Header)

- **Headers (Mid y Hot):** Volumen de 1.0 m³ cada uno.

- **Rol:** Actúan como capacitancias numéricas para estabilizar las ondas de presión y puntos de mezcla de entalpía.

4.1.3. Lazo de Control

Se utiliza un **Controlador Proporcional (P)** para regular la temperatura del Header Hot.

1. **Error de Temperatura (e):** $T_{\text{set}} - T_{\text{hot}}$.
2. **Potencia del Calefactor (Q):** $K_p \cdot e$.
3. **Ganancia (K_p):** $10000 \frac{\text{W}}{^\circ\text{C}}$.

Análisis de Resultados: Debido a que el control es puramente proporcional, el sistema presenta un **error de estado estacionario**. En la simulación, la temperatura se estabiliza cerca de los 39°C en lugar de los 60°C nominales, lo cual es físicamente correcto para este tipo de controlador ante una carga de flujo constante.

4.1.4. Estabilidad Numérica

El ejemplo utiliza el integrador **RK45** con un paso de sincronismo de 10 segundos. El motor adapta el paso interno para manejar la rigidez de las ecuaciones de momento de la bomba y los balances térmicos, garantizando una simulación libre de oscilaciones espurias.

5. Arquitectura de Integración con el Motor de Bloques

5.1. Integración con el Motor de Diagramas de Bloques

Este documento describe cómo la biblioteca termohidráulica (`thlib`) se acopla al motor de ejecución de bloques del proyecto `bloques`. La integración se basa en un esquema de **co-simulación de parámetros concentrados y distribuidos** utilizando una arquitectura de puertos de potencia.

5.1.1. El Paradigma de Acoplamiento (Esfuerzo-Flujo)

Para mantener la modularidad, los bloques termohidráulicos se dividen en dos categorías siguiendo la teoría de **Bond Graphs**:

1. Capacitancias (Headers, Tanks):

- **Entradas:** Caudal de masa (W) y Caudal de energía ($W \cdot h$) desde los puertos.
- **Salidas:** Estado termodinámico (Presión P , Entalpía h , Densidad ρ).
- **Rol:** Actúan como nodos de estado.

2. Resistencias (Pipes, Bombas, Válvulas):

- **Entradas:** Estados de los nodos fronterizos (P_{in} , P_{out} , h_{in} , h_{out}).
- **Salidas:** Caudales resultantes (W , $W \cdot h$).
- **Rol:** Actúan como conductores de flujo.

5.1.2. Mapeo de Puertos en el Sistema `bloques`

Cada componente de `thlib` debe envolverse en un `Block` de Rust que implemente la `trait SimulationBlock`.

```
pub struct ThPipeBlock {
    inner: Pipe1D,
}

impl SimulationBlock for ThPipeBlock {
    fn tick(&mut self, inputs: &[Value], outputs: &mut [Value], dt: f64) {
        // 1. Extraer presiones y entalpías de los puertos de entrada
        let p_in = inputs[0].as_f64();
        let h_in = inputs[1].as_f64();
        let p_out = inputs[2].as_f64();
        let h_out = inputs[3].as_f64();

        // 2. Ejecutar la física del componente
        let (w_in, w_out) = self.inner.tick(p_in, h_in, p_out, h_out, dt);

        // 3. Escribir caudales en los puertos de salida
        outputs[0] = Value::F64(w_in);
        outputs[1] = Value::F64(w_out);
    }
}
```

5.1.3. Orden de Ejecución y Estabilidad Numérica

Dado que el motor de bloques utiliza una ejecución secuencial, el orden de los bloques en el grafo es crítico para evitar el retraso de un paso de tiempo (z^{-1}) innecesario que podría desestabilizar las ondas de presión.

Orden Recomendado:

1. **Fuentes:** Actualizar señales de control (ej. velocidad de bomba, apertura de válvulas).
2. **Resistencias:** Calcular caudales basados en las presiones del paso anterior.
3. **Capacitancias:** Integrar masas y energías basadas en los nuevos caudales.
4. **Sinks:** Registrar resultados.

5.1.4. Manejo de la Red de Presión (Algebraic Loops)

En sistemas puramente incompresibles, la presión es una variable algebraica que se transmite instantáneamente. Sin embargo, en nuestro motor:

- Utilizamos **Capacitancias Numéricas** (Headers) para romper los lazos algebraicos y desacoplar las ecuaciones de momento.
- Cada conexión entre dos resistencias **debe** pasar obligatoriamente por un Header o Tanque.
- **Prohibición de Conexión Directa:** No se permite la conexión directa entre dos componentes de tipo resistencia (ej. Pipe1D -> Pipe1D). El orquestador de bloques debe validar la topología y emitir un error de compilación del grafo si detecta una conexión de este tipo, obligando al usuario a insertar un nodo capacitivo intermedio para garantizar la estabilidad física.

5.1.5. Sincronización Termodinámica Global

Para evitar inconsistencias, todos los bloques deben acceder a una única instancia (o implementación estática) de la librería de propiedades. En el sistema de bloques, esto se maneja mediante un Resource compartido:

```
// Ejemplo de acceso en el tick
let thermo = ctx.get_resource::<ThermoLibrary>();
let (p, h, t) = thermo.get_state_from_rho_u(rho, u);
```